

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ – CÂMPUS CORNÉLIO PROCÓPIO
ES45A – SISTEMAS DISTRIBUÍDOS – ES51
CURSO DE ENGENHARIA DE SOFTWARE

JOÃO LUCAS SILVA DE SOUZA
JOÃO VÍTOR OLIVEIRA ABREU
JOSÉ WYNDER ALVES HERNANDES
LUAN HENRIQUE DE ALMEIDA DOS SANTOS

Propagação de Fake News em Sistemas Paralelos e Distribuídos

CORNÉLIO PROCÓPIO

2026

Descrição do Problema

O projeto trata da simulação da propagação de fake news em uma população modelada por uma matriz bidimensional. Cada posição da matriz representa um indivíduo, e cada indivíduo pode estar em um de três estados específicos: ignorante, espalhador ou inativo. A propagação ocorre em gerações discretas e depende estritamente dos vizinhos mais próximos, seguindo a vizinhança de Moore, que engloba até oito vizinhos por célula.

O objetivo central deste trabalho foi transformar uma solução sequencial já existente em versões paralela e distribuída, mantendo o mesmo comportamento lógico original e viabilizando uma comparação rigorosa de desempenho e escalabilidade entre as três abordagens computacionais.

Explicação do Modelo Computacional

O modelo computacional adotado é inspirado no conceito de autômatos celulares. A população é estruturada em uma grade bidimensional (2D) e a evolução temporal do sistema ocorre em passos discretos denominados gerações. Em cada geração, o novo estado de cada indivíduo é calculado exclusivamente com base no estado atual da matriz inteira, sem alterar diretamente a geração em uso, garantindo a atomicidade da transição.

As regras de transição lógica obedecem aos seguintes critérios:

Ignorante para Espalhador: Um indivíduo ignorante se torna espalhador quando possui pelo menos uma quantidade mínima de vizinhos espalhadores, definida por um limiar de convencimento preestabelecido.

Espalhador para Inativo: Um espalhador torna-se inativo compulsoriamente na geração seguinte.

Inativo para Inativo: Um indivíduo inativo permanece inativo permanentemente, funcionando como um estado terminal na dinâmica de propagação.

Dessa forma, o modelo representa de maneira fiel a propagação local da informação de forma controlada, determinística e perfeitamente repetível.

Funcionamento da Solução Sequencial

Na versão sequencial, a simulação processa a matriz populacional inteira em um único fluxo de execução (single-thread). O ciclo de vida da execução compreende os seguintes passos:

Inicialização: A grade inicial é instanciada utilizando uma semente (seed) fixa para a geração pseudoaleatória, o que assegura a exata repetibilidade dos experimentos.

Processamento de Geração: A cada nova geração, o programa percorre a matriz de forma linear, contabiliza os vizinhos espalhadores de cada posição de acordo com a vizinhança de Moore e calcula o estado correspondente da próxima geração.

Consistência de Dados: O cálculo é escrito em uma nova estrutura de matriz auxiliar para preservar a consistência temporal da geração vigente. Ao término da iteração de todas as células, a matriz nova substitui a anterior.

Explicação das Versões Paralela e Distribuída

Versão Paralela (Multi-threading)

Na versão paralela, o processamento da matriz foi distribuído entre múltiplas threads concorrentes. Cada thread recebe uma faixa contínua de linhas e calcula estritamente a sua seção correspondente da próxima geração. Como todas as threads realizam operações de leitura sobre a mesma matriz da geração atual, a consistência é mantida. A sincronização ocorre ao final de cada geração através de barreiras lógicas, impedindo condições de corrida antes da atualização da matriz global.

Versão Distribuída (Mestre-Trabalhador via Sockets TCP)

Na versão distribuída, adotou-se o modelo de arquitetura mestre-trabalhador (Master-Worker) implementado por meio de sockets TCP. O processo mestre atua centralizando o controle: divide a matriz em blocos de linhas, transmite os segmentos de trabalho para os workers disponíveis, aguarda o retorno dos dados computados e realiza a montagem final da geração subsequente.

Dado que a transição de estado celular depende dos elementos limítrofes, o mestre implementa a técnica de linhas de fronteira (Halos). Isso consiste em enviar, junto com o bloco de linhas do worker, a linha imediatamente superior e a inferior vizinhas ao bloco. Desse modo, o worker obtém a informação necessária para computar corretamente a vizinhança de Moore nas bordas de seu domínio sem a necessidade de comunicação lateral entre processos

Divisão da Matriz e do Processamento

A decomposição do domínio foi realizada por meio de faixas contínuas de linhas da matriz (decomposição unidimensional). Esta abordagem simplifica o mapeamento de memória e reduz o overhead de particionamento.

Na Versão Paralela: As threads compartilham o mesmo espaço de endereçamento, lendo diretamente da matriz original e escrevendo em setores independentes da matriz destino.

Na Versão Distribuída: Há a necessidade de serialização dos dados. Cada worker recebe o seu bloco acrescido das linhas de halo. Após o processamento local, o bloco calculado retorna ao mestre, que reorganiza as faixas na ordem original para recompor o tecido global da matriz.

Configuração Experimental e Ambiente de testes

Ambiente de Hardware: Processador Intel® Core™ i7-13620H (13ª Geração, frequência base de 2.40 GHz, 10 núcleos físicos e 16 processadores lógicos) com 32.0 GB de memória RAM.

Ambiente de Software: Microsoft Windows 11 Pro (versão 25H2, compilação 26200.8655, 64 bits). Interpretador Python versão 3.14.6.

Condições de Execução: Execução nativa diretamente via terminal do Windows, sem a intermediação de IDEs, WSL, Docker ou ambientes virtuais, minimizando ruídos na coleta de tempos. Na versão distribuída, os workers foram emulados localmente via múltiplos processos utilizando a interface de loopback local (localhost).

As versões sequencial, paralela e distribuída foram executadas no mesmo ambiente computacional, de modo a permitir comparações diretas entre os tempos de execução. Na versão paralela, os experimentos variaram conforme a quantidade de threads ou processos definida em cada cenário. Na versão distribuída, os workers foram executados localmente, em múltiplos processos/terminais

na mesma máquina, simulando a distribuição da carga de trabalho sem o uso de múltiplos computadores físicos.

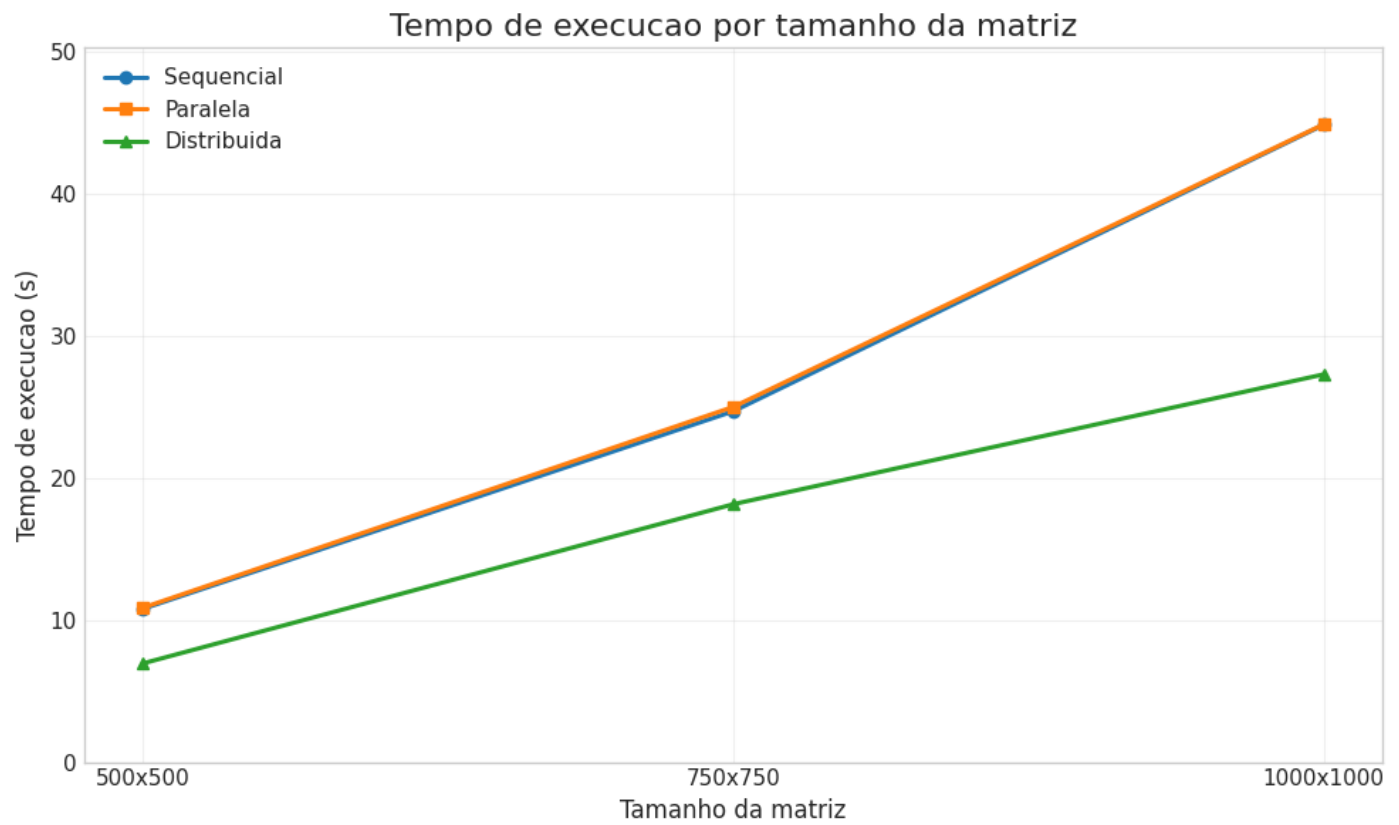
Tabelas e Gráficos

Os testes foram segmentados em grupos controlados para avaliar o impacto isolado de cada parâmetro do sistema.

Grupo 1: Variação do Tamanho da Matriz

Fixos: 50 gerações, 0.02 espalhadores iniciais, 4 Threads, 2 Workers.

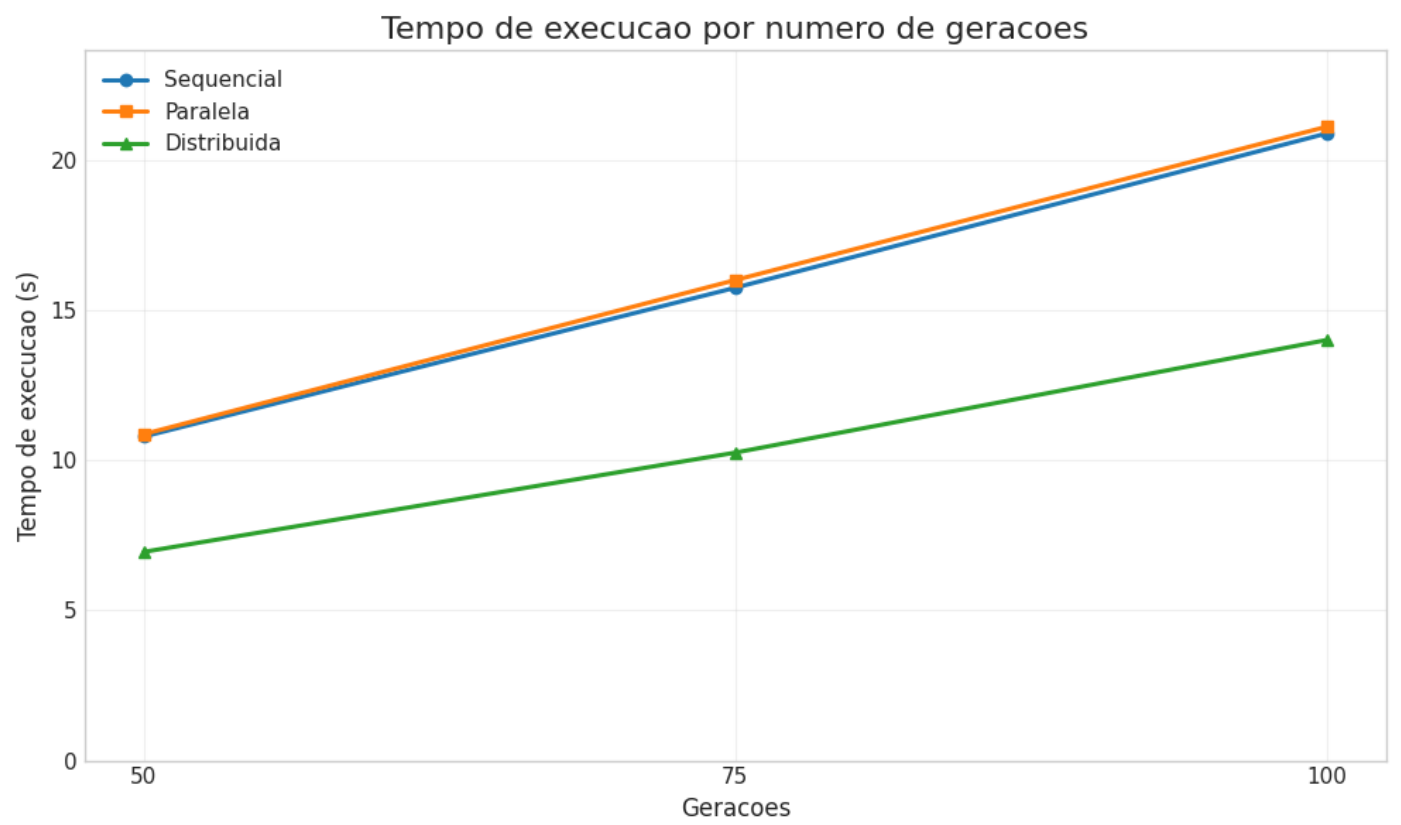
Dimensão	Sequencial (s)	Paralela (s)	Paralela Speedup	Paralela Eficiência	Distribuída (s)	Distribuída Speedup	Distribuída Eficiência
500 x 500	10.7919	10.8726	0.99	0.25	6.9492	1.55	0.78
750 x 750	24.7088	25.0178	0.99	0.25	18.1619	1.36	0.68
1000 x 1000	44.8833	44.9163	1.00	0.25	27.3062	1.64	0.82



Grupo 2: Variação do Número de Gerações

Fixos: Matriz 500 x 500, 0.02 espalhadores iniciais, 4 Threads, 2 Workers.

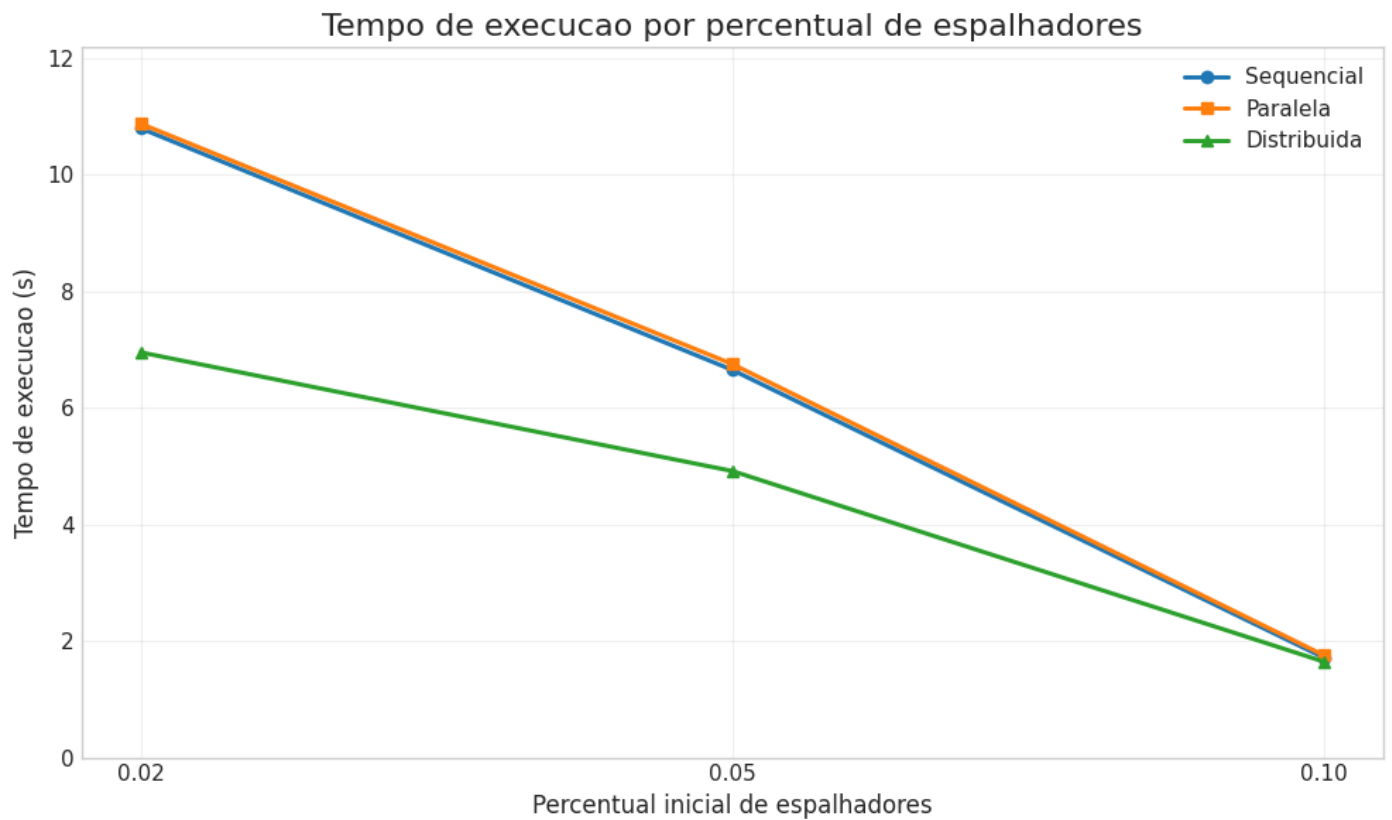
Gerações	Sequencial (s)	Paralela (s)	Paralela Speedup	Distribuída (s)	Distribuída Speedup
50	10.7919	10.8726	0.99	6.9492	1.55
75	15.7534	16.0088	0.98	10.2589	1.54
100	20.8989	21.1225	0.99	14.0136	1.49



Grupo 3: Variação do Percentual Inicial de Espalhadores

Fixos: Matriz 500 x 500, 50 gerações, 4 Threads, 2 Workers.

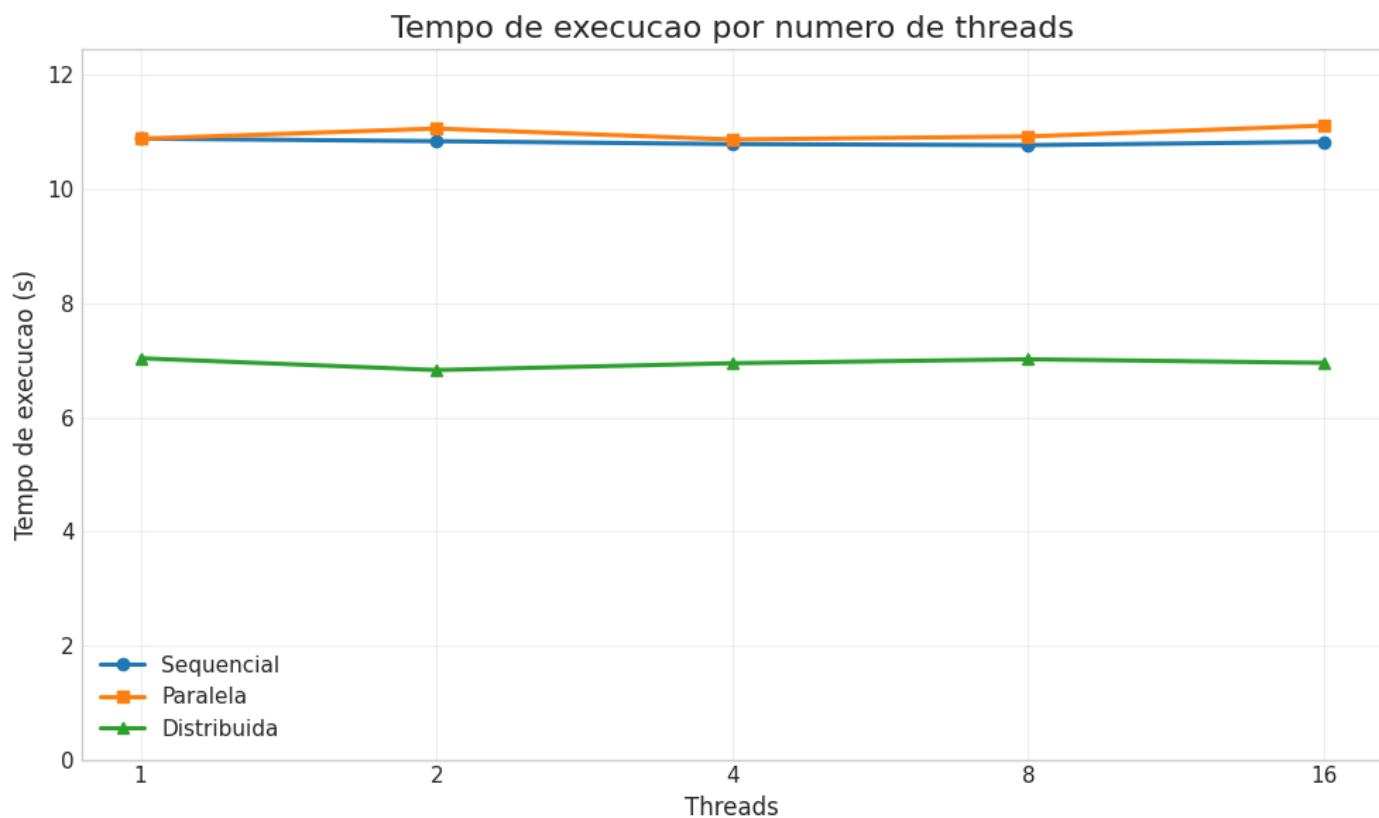
Densidade Inicial	Sequencial (s)	Paralela (s)	Paralela Speedup	Distribuída (s)	Distribuída Speedup
0.02	10.7919	10.8726	0.99	6.9492	1.55
0.05	6.6469	6.7476	0.99	4.9135	1.35
0.10	1.7101	1.7514	0.98	1.6417	1.04



Grupo 4: Variação do Número de Threads (Escalabilidade Paralela)

Fixos: Matriz 500 x 500, 50 gerações, 0.02 espalhadores, 2 Workers (referência distribuída estável).

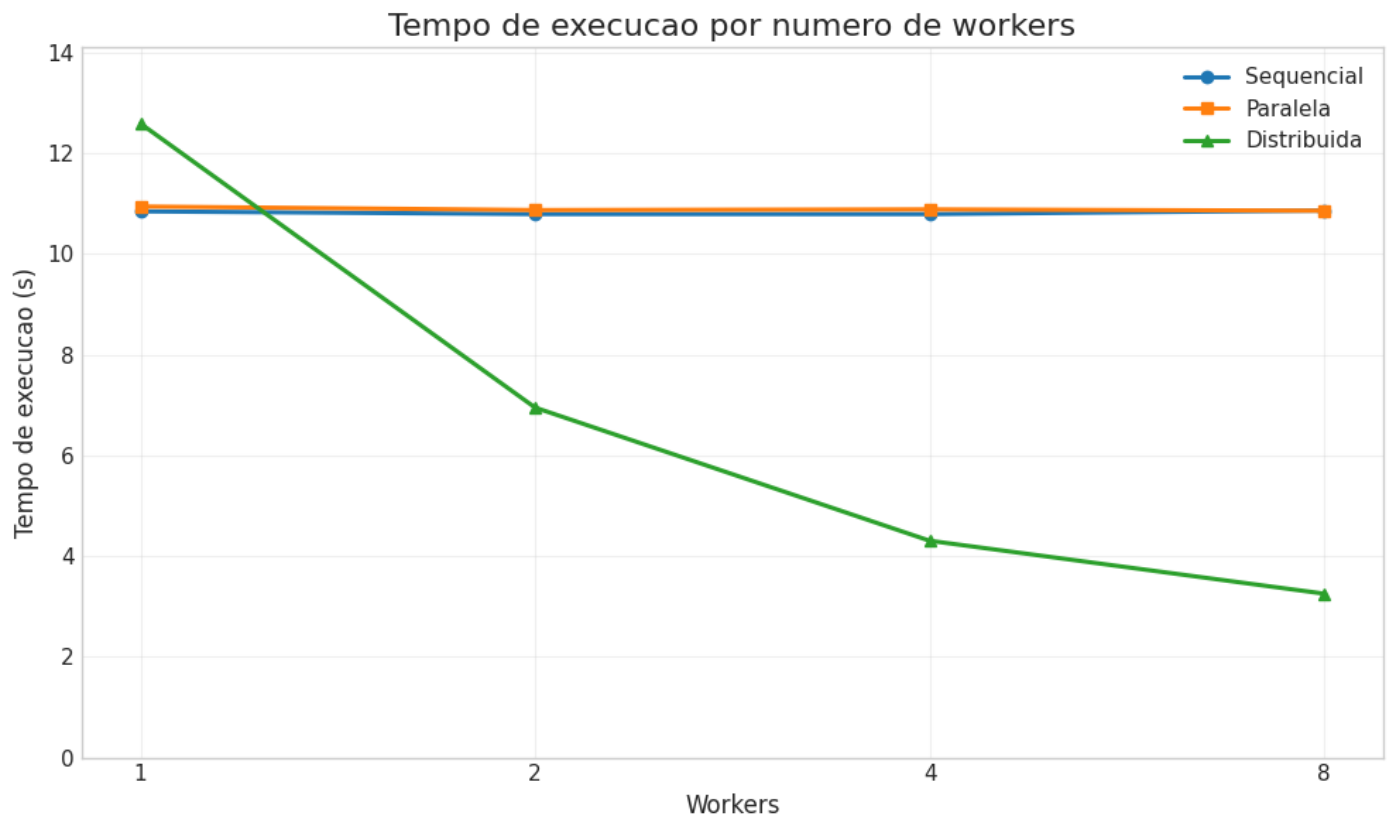
Threads	Sequencial (s)	Paralela (s)	Speedup Paralelo	Eficiência Paralela
1 Thread	10.8888	10.8851	1.00	1.00
2 Threads	10.8429	11.0646	0.98	0.49
4 Threads	10.7919	10.8726	0.99	0.25
8 Threads	10.7730	10.9264	0.99	0.12
16 Threads	10.8311	11.1162	0.97	0.06



Grupo 5: Variaão do Número de Workers (Escalabilidade Distribuïda)

Fixos: Matriz 500 x 500, 50 geraões, 0.02 espalhadores, 4 Threads.

Workers	Sequencial (s)	Distribuïda (s)	Speedup Distribuïdo	Eficiência Distribuïda
1 Worker	10.8457	12.5893	0.86	0.86
2 Workers	10.7919	6.9492	1.55	0.78
4 Workers	10.7915	4.3049	2.51	0.63
8 Workers	10.8633	3.2535	3.34	0.42



Análise Crítica: Speedup e Eficiência

O Comportamento da Versão Paralela e o Impacto do GIL

A análise dos dados do Grupo 4 revela um fenômeno crítico clássico do ecossistema Python: o tempo de execução da versão paralela permaneceu estagnado próximo ao tempo sequencial (~10.8s), fazendo com que a eficiência despencasse de 1.00 (com 1 thread) para insignificantes 0.06 (com 16 threads). Isso se deve diretamente ao GIL (Global Interpreter Lock) do interpretador padrão do Python (CPython).

Como a tarefa de atualização das células do autômato celular é puramente computacional (CPU-bound) e não envolve operações de I/O bloqueantes, o GIL impede que múltiplas threads executem bytecode Python simultaneamente em múltiplos núcleos de CPU físicos.

Consequentemente, o paralelismo com a biblioteca threading tornou-se meramente concorrente em nível lógico, adicionando o overhead de chaveamento de contexto sem ganho de desempenho real.

O Sucesso da Abordagem Distribuïda

Em contrapartida, a versão distribuïda obteve excelente ganho de escala (Grupo 5). Ao isolar a computação em processos independentes comunicando-se via sockets TCP, cada worker obteve sua própria instância do interpretador Python e seu próprio GIL. Isso permitiu o paralelismo real utilizando os múltiplos núcleos do processador Intel Core i7-13620H.

Nota-se que com 1 Worker a execução foi mais lenta que a sequencial (12.58s vs 10.84s) devido ao overhead inicial de comunicação e rede local. Todavia, ao aumentar para 8 Workers, o tempo reduziu para 3.2535 segundos, atingindo um Speedup expressivo de 3.34x.

Curiosidade Lógica na Variação de Espalhadores

No Grupo 3, constatou-se que o aumento na densidade inicial de espalhadores reduziu drasticamente o tempo de processamento global (de 10.79s para 1.71s). Isso decorre da

dinâmica intrínseca do modelo: uma alta concentração inicial de espalhadores causa uma rápida saturação da matriz, levando a população ao estado terminal inativo de forma acelerada, atingindo o critério de parada por ausência de espalhadores muito antes do limite das 50 gerações configuradas.

Dificuldades Encontradas

- **Garantia de Corretude:** Manter o comportamento lógico idêntico entre as versões sequencial, paralela e distribuída exigiu centralizar as regras de transição em um modelo compartilhado, evitando divergências nos resultados.
- **Sincronização e Concorrência:** O cálculo de cada geração depende de uma cópia estável do estado anterior. Na versão paralela, isso demandou o uso de matrizes auxiliares e barreiras de sincronização entre as *threads* para evitar condições de corrida.
- **Comunicação e Overhead:** A versão distribuída exigiu o envio constante de linhas de fronteira (*halos*) entre mestre e *workers*. Essa coordenação gerou um custo de comunicação alto e maior sensibilidade a falhas.
- **Calibração de Parâmetros:** Inicialmente, as simulações estabilizavam rápido demais, gerando tempos de execução irrelevantes para análise. Foi necessário ajustar exaustivamente o tamanho da matriz e as taxas de contágio para obter cenários de teste consistentes.
- **Divergência entre Plataformas:** A análise de desempenho revelou limitações tecnológicas distintas: em Python, o ganho das *threads* foi limitado pelo GIL em tarefas CPU-bound, enquanto Java apresentou um ganho de escala real.
- **Simplicidade Acadêmica:** O maior desafio prático foi evitar o excesso de complexidade arquitetural, optando por soluções diretas — como a divisão estática da matriz por linhas — para priorizar a clareza e a facilidade de apresentação.

Melhorias Implementadas

- **Modelo Estocástico (Ceticismo):** A infecção deixou de ser determinística e passou a incluir uma probabilidade aleatória de convencimento, permitindo que células resistam à notícia e gerem "bolsões de isolamento" na matriz.
- **Influenciadores Digitais (Super-Spreaders):** Inclusão de um estado com peso multiplicado no cálculo de contágio da vizinhança e maior tempo de permanência ativa antes da exaustão.
- **Contas Automatizadas (Bots):** Criação de um estado de propagação perpétua que nunca transita para o modo inativo, servindo como uma fonte eterna de contágio que eleva o tempo de execução e estressa o processamento paralelo.
- **Agências de Fact-Checking (Resistência):** Mecânica inversa aos influenciadores, onde nós verificadores possuem peso negativo na vizinhança, mitigando o avanço das fake news e protegendo nós saudáveis.
- **Métricas Epidemiológicas Avançadas:** Expansão da coleta de dados para registrar o comportamento histórico da simulação, identificando o pico exato de infecção e a geração de estabilização do sistema.
- **Geração Gráfica Automatizada:** Integração de scripts que geram e exportam automaticamente as curvas de contágio em formato .png ao final da execução, eliminando o trabalho manual com planilhas externas.

Referências Bibliográficas

- SILBERSCHATZ, Abraham; GALVIN, Peter Baer; GAGNE, Greg. *Sistemas Operacionais*. 9. ed. Rio de Janeiro: LTC, 2015.
- TANENBAUM, Andrew S.; VAN STEEN, Maarten. *Sistemas Distribuídos: Princípios e Paradigmas*. 2. ed. São Paulo: Pearson, 2007.
- PYTHON SOFTWARE FOUNDATION. *The Python Standard Library: threading & socket modules*. Disponível em: <<https://docs.python.org/3/>>. Acesso em: 2026.

Contribuição Individual

1. João Lucas: Implementou os códigos e efetuou pesquisas.
2. José Wynder: Implementou os códigos e efetuou pesquisas.
3. Luan Henrique: Executou as simulações, coletou os tempos e calculou o speedup e a eficiência.
4. João Vítor: Redigiu o relatório final em PDF e gerou os gráficos

